



मुंबई विद्यापीठ
University of Mumbai
Re-accredited with A++ Grade
(CGPA 3.65) by NAAC (3rd Cycle 2021)

University of Mumbai

DEPARTMENT OF COMPUTER SCIENCE

JOURNAL

M. Sc. (Computer Science) (NEP) Semester-III

2025-2026

Web3 Technologies

Submitted by
Dheeraj Vinod Giri

Seat No.
MU0027920240004601



मुंबई विद्यापीठ
University of Mumbai
Re-accredited with A++ Grade
(CGPA 3.65) by NAAC (3rd Cycle 2021)

University of Mumbai

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the work entered in this journal was done in the Department of Computer Science, University of Mumbai by Mr. Dheeraj Vinod Giri Seat No.0027920240004601 for the course of **M.Sc. (Computer Science) (NEP) Semester-III** during the academic year **2025-26** in a satisfactory manner.

Subject In-charge
Department of Computer Science

Head
Department of Computer Science

External Examiner

INDEX

Practical No.	Name of the Practical	Page No.	Date	Signature
1	Create and deploy a block chain network using Hyperledger Fabric SDK for Java	1		
2	Interact with a block chain network. Execute transactions and requests against a block chain network by creating an app to test the network and its rules	3		
3	Deploy an asset-transfer app using block chain. Learn app development within a Hyperledger Fabric network.	5		
4	Use block chain to track fitness club rewards..	7		
5	Build a web app that uses Hyperledger Fabric to track and trace member rewards.	10		
6	Car auction network: A Hello World example with Hyperledger Fabric Node SDK and IBM Block chain Starter Plan. Use Hyperledger Fabric to invoke chaincode while storing results and data in the starter plan	15		
7	Develop an IoT asset tracking app using Block chain. Use an IoT asset tracking device to improve a supply chain by using Block chain, IoT devices, and Node-RED.	22		
8	Create a global finance block chain application with IBM Block chain Platform Extension for VS Code. Develop a Node.js smart contract and web app for a Global Finance with block chain use case	26		
9	Develop a voting application using Hyperledger and Ethereum.	32		

PRACTICAL 1

Aim : Create and deploy a block chain network using Hyperledger Fabric SDK for Java

Objective:

- To simulate the creation and deployment of a blockchain network using Hyperledger Fabric SDK for Java on JDoodle, without installing any local software. This simulation provides conceptual understanding of peers, ledger, transactions, and chaincode execution in Fabric.

Procedure (JDoodle Steps):

Step 1: Open <https://www.jdoodle.com/> and choose '**Java**' as the language.

Step 2: Copy the below simulation code into the editor.

Step 3: Click on '**Execute**' to run the simulation.

Step 4: Observe the blockchain ledger creation, transaction endorsement, and commit process.

Code :

```
import java.util.*;
class Block {
    int index;
    String data;
    String prevHash;
    String hash;
    String timestamp;
    Block(int index, String data, String prevHash) {
        this.index = index;
        this.data = data;

        this.prevHash = prevHash;
        this.timestamp = new Date().toString();
        this.hash = Integer.toString((data + prevHash + index +
timestamp).hashCode());
    }
}
class Ledger {
    List<Block> chain = new ArrayList<>();

    public void addBlock(String data) {
        String prevHash = chain.isEmpty() ? "0" : chain.get(chain.size() - 1).hash;
        Block block = new Block(chain.size(), data, prevHash);
        chain.add(block);
    }
    public void printChain() {
        for (Block b : chain) {
            System.out.println("Block " + b.index + " [Data: " + b.data + ", Hash: " +
b.hash + ", Time: " + b.timestamp + "]);
        }
    }
}
```

```

public class FabricJavaSimulation {
    public static void main(String[] args) {
        System.out.println("=== Hyperledger Fabric Java SDK Simulation ===");

        Ledger ledger = new Ledger();
        ledger.addBlock("Genesis Block: Network Initialized");
        ledger.addBlock("Peer1 endorses transaction: Add asset A");
        ledger.addBlock("Peer2 endorses transaction: Transfer asset A");
        ledger.addBlock("Peer3 commits transaction: Ledger Updated");

        System.out.println("\nBlockchain Ledger State:");
        ledger.printChain();
    }
}

```

Output :

```

=== Hyperledger Fabric Java SDK Simulation ===

```

Blockchain Ledger State:

Block 0 [Data: Genesis Block: Network Initialized, Hash: 11323849, Time: Sat Jan 03 14:09:53 GMT 2026]

Block 1 [Data: Peer1 endorses transaction: Add asset A, Hash: 907688592, Time: Sat Jan 03 14:09:53 GMT 2026]

Block 2 [Data: Peer2 endorses transaction: Transfer asset A, Hash: -1453774001, Time: Sat Jan 03 14:09:53 GMT 2026]

Block 3 [Data: Peer3 commits transaction: Ledger Updated, Hash: 1309031391, Time: Sat Jan 03 14:09:53 GMT 2026]

PRACTICAL 2

Aim : Interact with a block chain network. Execute transactions and requests against a block chain network by creating an app to test the network and its rules.

Objective :

- To simulate interaction with a blockchain network by executing transactions and requests through a Java-based application.
The app mimics how the Hyperledger Fabric Java SDK performs read/write operations on a ledger, validates transactions, and enforces rules.

Code :

```
import java.util.*;

class Transaction {
    String txID;
    String assetID;
    String action;
    String user;
    String status;

    Transaction(String txID, String assetID, String action, String user) {
        this.txID = txID;
        this.assetID = assetID;
        this.action = action;
        this.user = user;
        this.status = "PENDING";
    }
}

class BlockchainNetwork {
    Map<String, String> ledger = new HashMap<>();
    List<Transaction> history = new ArrayList<>();

    // Simulate endorsement rule

    boolean endorseTransaction(Transaction tx) {
        return !tx.assetID.isEmpty() && !tx.user.isEmpty();
    }

    // Submit transaction
    void submitTransaction(Transaction tx) {
        if (endorseTransaction(tx)) {
            if (tx.action.equals("CREATE")) {
                ledger.put(tx.assetID, tx.user);
                tx.status = "COMMITTED";
            } else if (tx.action.equals("TRANSFER")) {
                if (ledger.containsKey(tx.assetID)) {
                    ledger.put(tx.assetID, tx.user);
                    tx.status = "COMMITTED";
                } else {
```

```

        tx.status = "REJECTED (Asset not found)";
    }
    } else if (tx.action.equals("DELETE")) {
        ledger.remove(tx.assetID);
        tx.status = "COMMITTED";
    }
    } else {
        tx.status = "REJECTED (Endorsement failed)";
    }
    history.add(tx);
}

void queryLedger() {
    System.out.println("\n--- Current Ledger State ---");
    ledger.forEach((a, o) -> System.out.println("Asset: " + a + " | Owner: " + o));
}

void viewHistory() {
    System.out.println("\n--- Transaction History ---");
    for (Transaction t : history) {
        System.out.println(t.txID + " | " + t.action + " | " + t.assetID + " | " + t.user +
" | " + t.status);
    }
}

}

public class FabricTransactionApp {
    public static void main(String[] args) {
        System.out.println("=== Hyperledger Fabric Java SDK Transaction Simulation
===");
        BlockchainNetwork network = new BlockchainNetwork();

        Transaction t1 = new Transaction("TX001", "Asset1", "CREATE", "Alice");
        Transaction t2 = new Transaction("TX002", "Asset1", "TRANSFER", "Bob");
        Transaction t3 = new Transaction("TX003", "Asset1", "DELETE", "Admin");
        network.submitTransaction(t1);
        network.submitTransaction(t2);
        network.submitTransaction(t3);
        network.queryLedger();
        network.viewHistory();
    }
}

```

Output :

```

=== Hyperledger Fabric Java SDK Transaction Simulation ===

--- Current Ledger State ---

--- Transaction History ---
TX001 | CREATE | Asset1 | Alice | COMMITTED
TX002 | TRANSFER | Asset1 | Bob | COMMITTED
TX003 | DELETE | Asset1 | Admin | COMMITTED

```

PRACTICAL 3

Aim : Deploy an asset-transfer app using block chain. Learn app development within a Hyperledger Fabric network.

Objective:

- To understand how to deploy and interact with an asset-transfer application using Hyperledger Fabric Java SDK, and to simulate the process online using Java classes and collections to represent the blockchain network components.

Code :

```
import java.util.*;

class Ledger {
    private Map<String, Asset> assets = new HashMap<>();

    // Create a new asset
    public void createAsset(String assetId, String owner, int value) {
        if (assets.containsKey(assetId)) {
            System.out.println("Asset already exists!");
        } else {
            assets.put(assetId, new Asset(assetId, owner, value));
            System.out.println("Asset created: " + assetId);
        }
    }

    // Read asset
    public void readAsset(String assetId) {
        Asset a = assets.get(assetId);
        if (a != null)
            System.out.println("Asset ID: " + a.assetId + ", Owner: " + a.owner + ", Value: " + a.value);
        else
            System.out.println("Asset not found!");
    }

    // Transfer asset ownership
    public void transferAsset(String assetId, String newOwner) {
        Asset a = assets.get(assetId);
        if (a != null) {
            a.owner = newOwner;
            System.out.println("Asset transferred to: " + newOwner);
        }
        else
        {

```

```

        System.out.println("Asset not found!");
    }
}

// Display all assets
public void displayLedger() {
    System.out.println("\n=== Blockchain Ledger ===");
    for (Asset a : assets.values()) {
        System.out.println(a.assetId + " | " + a.owner + " | " + a.value);
    }
}
}

class Asset {
    String assetId;
    String owner;
    int value;

    Asset(String assetId, String owner, int value) {
        this.assetId = assetId;
        this.owner = owner;
        this.value = value;
    }
}

public class AssetTransferApp {
    public static void main(String[] args) {
        Ledger ledger = new Ledger();

        // Simulated transactions
        ledger.createAsset("asset1", "Alice", 1000);
        ledger.createAsset("asset2", "Bob", 500);
        ledger.readAsset("asset1");
        ledger.transferAsset("asset1", "Charlie");
        ledger.displayLedger();
    }
}

```

Output :

```

Asset created: asset1
Asset created: asset2
Asset ID: asset1, Owner: Alice, Value: 1000
Asset transferred to: Charlie

=== Blockchain Ledger ===
asset1 | Charlie | 1000
asset2 | Bob | 500

```

PRACTICAL 4

Aim : Use block chain to track fitness club rewards.

Objective :

To design and simulate a blockchain-based Fitness Club Rewards Tracking System using Java to understand :

- Asset creation and ownership
- Transaction recording (reward earning/redeeming)
- Immutable ledger updates (simulated)
- Mapping of Hyperledger Fabric components to Java objects

Code :

```
import java.util.*;
// Asset class representing a club member
class Member {
    String memberId;
    String name;
    int rewardPoints;

    Member(String memberId, String name) {
        this.memberId = memberId;
        this.name = name;
        this.rewardPoints = 0;
    }
}

// Transaction class
class Transaction {
    String txId;
    String memberId;
    String type;
    int points;
    Date timestamp;

    Transaction(String txId, String memberId, String type, int points) {
        this.txId = txId;
        this.memberId = memberId;
        this.type = type;
        this.points = points;
        this.timestamp = new Date();
    }

    @Override
    public String toString() {
        return txId + " | " + memberId + " | " + type + " | " + points + " pts | " +
            timestamp;
    }
}

// Ledger simulating blockchain
```

```

class RewardLedger {
Map<String, Member> members = new HashMap<>();
List<Transaction> transactions = new ArrayList<>();

// Register new member
public void registerMember(String id, String name) {
if (members.containsKey(id)) {
System.out.println("⚠ Member already exists!");
} else {
members.put(id, new Member(id, name));
System.out.println("✅ Registered new member: " + name);
}
}

// Earn reward points
public void earnPoints(String id, int points) {
Member m = members.get(id);
if (m != null) {
m.rewardPoints += points;
Transaction tx = new Transaction(UUID.randomUUID().toString(), id,
"EARN", points);
transactions.add(tx);
System.out.println("👤 " + m.name + " earned " + points + " points!");
} else {
System.out.println("⚠ Member not found!");
}
}

// Redeem reward points
public void redeemPoints(String id, int points) {
Member m = members.get(id);
if (m != null) {
if (m.rewardPoints >= points) {
m.rewardPoints -= points;
Transaction tx = new Transaction(UUID.randomUUID().toString(), id,
"REDEEM", points);
transactions.add(tx);
System.out.println("👤 " + m.name + " redeemed " + points + " points!");
} else {
System.out.println("⚠ Not enough points!");
}
} else {
System.out.println("⚠ Member not found!");
}
}

// Display ledger
public void displayLedger() {
System.out.println("\n=== 📖 Blockchain Ledger (Transactions) ===");
}
}

```

```

for (Transaction t : transactions) {
    System.out.println(t);
}

// Display member summary
public void displayMembers() {
    System.out.println("\n=== \Member Status ===");
    for (Member m : members.values()) {
        System.out.println(m.memberId + " | " + m.name + " | Points: " + m.rewardPoints);
    }
}

public class FitnessClubRewards {
    public static void main(String[] args) {
        RewardLedger ledger = new RewardLedger();

        // Register members
        ledger.registerMember("M001", "Alice");
        ledger.registerMember("M002", "Bob");

        // Simulated blockchain transactions
        ledger.earnPoints("M001", 50);
        ledger.earnPoints("M002", 30);
        ledger.redeemPoints("M001", 20);
        ledger.earnPoints("M001", 15);

        // Display results
        ledger.displayMembers();
        ledger.displayLedger();
    }
}

```

Output :

 Registered new member: Alice
 Registered new member: Bob
 Alice earned 50 points!
 Bob earned 30 points!
 ~~Alice~~ redeemed 20 points!
 Alice earned 15 points!

=== \Member Status ===
 M002 | Bob | Points: 30
 M001 | Alice | Points: 45

=== ^ Blockchain Ledger (Transactions) ===
 79f73776-d8d8-454a-879f-2505883cd8a4 | M001 | EARN | 50 pts | Sat Jan 03 09:27:46 GMT 2026
 8ed71bda-603f-4ea3-a31d-32a9629e6d9c | M002 | EARN | 30 pts | Sat Jan 03 09:27:46 GMT 2026
 eba4ee7f-3f88-4684-a26c-c42ebc9feaba | M001 | REDEEM | 20 pts | Sat Jan 03 09:27:46 GMT 2026
 a63ae043-d10f-4b2a-9e3a-6c3e63e7711d | M001 | EARN | 15 pts | Sat Jan 03 09:27:46 GMT 2026

PRACTICAL 5

Aim : Build a web app that uses Hyperledger Fabric to track and trace member rewards .

Objective :

- Track and trace rewards earned/redeemed by members.
- Simulate Hyperledger Fabric functionality: ledger, transactions, smart contracts (chaincode), identities.
- Develop a web app with a frontend + backend + simulated blockchain ledger.

Tools :

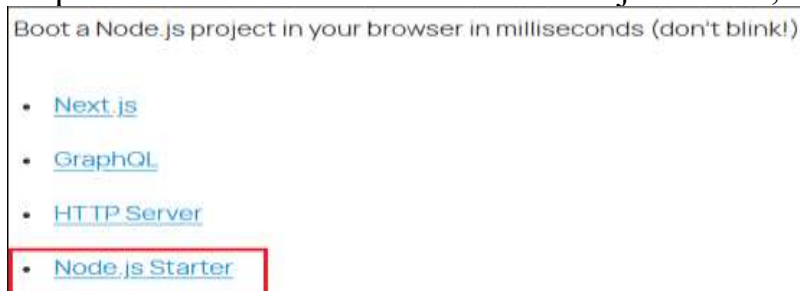
- Node.js + Express (backend)
- HTML/CSS/JS (frontend)
- UUID library for unique transaction IDs
- Free online IDE: StackBlitz,

Execution Steps :

Step 1 : Go to <https://stackblitz.com/>, scroll and click on this.



Step 2 : Scroll down and click on “Node.js Starter”, it will open a new IDE.



Step 3 : Create a new “Server.js” file

Server.js

```
console.log(`Hello Node.js v${process.versions.node}`);
const express = require('express');
const bodyParser = require('body-parser');
const { v4: uuidv4 } = require('uuid');
const app = express();
const PORT = 3000;

app.use(bodyParser.json());
```

```

app.use(express.static('public'));

// Simulated ledger
let members = {};
let transactions = [];

// Register member
app.post('/api/register', (req, res) => {
  const { memberId, name } = req.body;
  if (members[memberId]) return res.status(400).send({ msg: "Member exists" });
  members[memberId] = { memberId, name, points: 0 };
  res.send({ msg: `Member ${name} registered!` });
});

// Earn points
app.post('/api/earn', (req, res) => {
  const { memberId, points } = req.body;
  const member = members[memberId];
  if (!member) return res.status(404).send({ msg: "Member not found" });
  member.points += points;
  const tx = { txId: uuidv4(), memberId, type: 'EARN', points, timestamp: new Date() };
  transactions.push(tx);
  res.send(tx);
});

// Redeem points
app.post('/api/redeem', (req, res) => {
  const { memberId, points } = req.body;
  const member = members[memberId];
  if (!member) return res.status(404).send({ msg: "Member not found" });
  if (member.points < points) return res.status(400).send({ msg: "Not enough points" });
  member.points -= points;
  const tx = { txId: uuidv4(), memberId, type: 'REDEEM', points, timestamp: new Date() };
  transactions.push(tx);
  res.send(tx);
});

// Get member info
app.get('/api/member/:id', (req, res) => {

  const member = members[req.params.id];
  if (!member) return res.status(404).send({ msg: "Member not found" });
  res.send(member);
});

// Get all transactions
app.get('/api/ledger', (req, res) => {
  res.send(transactions);
});
app.listen(PORT, () => console.log(`Server running at http://localhost:\${PORT}`));

```

Step 4 : Now Create a **public folder** and inside it, create “**Index.html**” file

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Fitness Club Rewards</title>
    <style>
      body {
        font-family: Arial;
        margin: 20px;
      }
      input,
      button {
        margin: 5px;
      }
    </style>
  </head>
  <body>
    <h2>Fitness Club Rewards Tracker</h2>

    <h3>Register Member</h3>
    <input id="regId" placeholder="Member ID" />
    <input id="regName" placeholder="Name" />
    <button onclick="register()">Register</button>
    <p id="regMsg"></p>

    <h3>Earn Points</h3>
    <input id="earnId" placeholder="Member ID" />
    <input id="earnPoints" type="number" placeholder="Points" />
    <button onclick="earnPoints()">Earn</button>

    <h3>Redeem Points</h3>
    <input id="redeemId" placeholder="Member ID" />
    <input id="redeemPoints" type="number" placeholder="Points" />
    <button onclick="redeemPoints()">Redeem</button>

    <h3>View Ledger</h3>
    <button onclick="viewLedger()">Show Ledger</button>
    <pre id="ledger"></pre>

    <script>
      async function register() {
        const id = document.getElementById('regId').value;
        const name = document.getElementById('regName').value;
        const res = await fetch('/api/register', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ memberId: id, name: name })
        });

        const data = await res.json();
        document.getElementById('regMsg').innerText =
          data.msg || JSON.stringify(data);
      }
    </script>
  </body>
</html>
```

```

    }

    async function earnPoints() {
      const id = document.getElementById('earnId').value;
      const points = parseInt(document.getElementById('earnPoints').value);
      const res = await fetch('/api/earn', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ memberId: id, points }),
      });
      const data = await res.json();
      alert("Transaction: " + JSON.stringify(data));
    }

    async function redeemPoints() {
      const id = document.getElementById('redeemId').value;
      const points = parseInt(document.getElementById('redeemPoints').value);
      const res = await fetch('/api/redeem', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ memberId: id, points }),
      });
      const data = await res.json();
      alert("Transaction: " + JSON.stringify(data));
    }

    async function viewLedger() {
      const res = await fetch('/api/ledger');
      const data = await res.json();
      document.getElementById('ledger').innerText = JSON.stringify(
        data,
        null,
        2
      );
    }
  }
</script>
</body>
</html>

```

Step 5 : Install required dependencies :

- npm install express body-parser uuid

Step 6 : Start the server

- In the terminal run “**node server.js**” → you will get a live URL.
- Open the URL → interact with the app: register, earn, redeem, view ledger.

Output :

Fitness Club Rewards Tracker

Register Member

Register

Member abc registered!

Earn Points

Earn

Redeem Points

Redeem

Fitness Club Rewards Tracker

Register Member

Register

Earn Points

Earn

Redeem Points

Redeem

View Ledger

Show Ledger

View Ledger

Show Ledger

```
[
  {
    "txId": "25ed5ce0-a7df-48d2-9eaf-e976a03d5e59",
    "memberId": "123",
    "type": "EARN",
    "points": 4,
    "timestamp": "2025-12-06T04:00:04.428Z"
  },
  {
    "txId": "d34d6671-8eca-482d-a6b9-5079db303d74",
    "memberId": "123",
    "type": "REDEEM",
    "points": 4,
    "timestamp": "2025-12-06T04:00:16.977Z"
  }
]
```

PRACTICAL 6

Aim : Car auction network: A Hello World example with Hyperledger Fabric Node SDK and IBM Block chain Starter Plan. Use Hyperledger Fabric to invoke chaincode while storing results and data in the starter plan

Objective :

- To develop and deploy a Car Auction Network using Hyperledger Fabric on the IBM Blockchain Starter Plan.
- To create and execute a Node.js application (on StackBlitz) that connects to the IBM Fabric network through the Fabric Node SDK.
- To invoke chaincode transactions, query data, and store results on the IBM-managed blockchain ledger.

File Structure :

```
/ (root)
├── app.js
├── carauction.js
├── ledger.json
├── package.json
├── public/
│   └── index.html
```

Package.json

```
{
  "name": "car-auction-simulation",
  "version": "1.0.0",
  "description": "Simulated Car Auction Blockchain using Fabric Node SDK structure with HTML frontend",
  "main": "app.js",
  "type": "module",
  "dependencies": {
    {
      "express": "^4.19.2",
      "body-parser": "^1.20.2"
    }
  }
}
```

App.js

```
import express from "express";
import bodyParser from "body-parser";
import { CarAuctionContract } from "./carauction.js";

const app = express();
app.use(bodyParser.json());
app.use(express.static("public"));
const contract = new CarAuctionContract();

// Initialize ledger
(async () => {
  await contract.initLedger();
})();

// API ROUTES
app.get("/api/cars", async (req, res) => {
  res.json(await contract.queryAllCars());
});

app.post("/api/register", async (req, res) => {
  const { carId, model, owner } = req.body;
  try {
    const result = await contract.registerCar(carId, model, owner);
    res.json({ message: result });
  } catch (e) {
    res.status(400).json({ error: e.message });
  }
});

app.post("/api/bid", async (req, res) => {
  const { carId, bidder, amount } = req.body;
  try {
    const updated = await contract.placeBid(carId, bidder, amount);
    res.json(updated);
  } catch (e) {
    res.status(400).json({ error: e.message });
  }
});

app.get("/api/highestBid", async (req, res) => {
  res.json(await contract.getHighestBid());
});

// Root route
app.get("/", (req, res) => {
  res.sendFile("/index.html", { root: "public" });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () =>
  console.log(`🚀 Car Auction app running on port ${PORT}`)
);
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>

Car Auction Blockchain Simulation</title>
<style>
body { font-family: Arial; margin: 40px; background: #f4f6f8; }
h1 { color: #007bff; }

    section { background: white; padding: 20px; margin-bottom: 20px; border-radius:
10px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
    button { background: #007bff; color: white; border: none; padding: 8px 15px;
border-radius: 5px; cursor: pointer; }
    input { margin: 5px; padding: 6px; }
    pre { background: #eee; padding: 10px; border-radius: 6px; }
</style>
</head>
<body>
<h1>

Car Auction Blockchain Simulation</h1>

<section>
    <h3>View All Cars</h3>
    <button onclick="getCars()">Show Cars</button>
    <pre id="cars"></pre>
</section>

<section>
    <h3>Register New Car</h3>
    <input id="newId" placeholder="Car ID">
    <input id="newModel" placeholder="Model">
    <input id="newOwner" placeholder="Owner">
    <button onclick="registerCar()">Register</button>
    <pre id="registerResult"></pre>
</section>

<section>
    <h3>Place a Bid</h3>
    <input id="bidCarId" placeholder="Car ID">
    <input id="bidder" placeholder="Bidder Name">
    <input id="bidAmount" placeholder="Amount">
    <button onclick="placeBid()">Submit Bid</button>
    <pre id="bidResult"></pre>
</section>
```

```

<section>
  <h3>Highest Bid</h3>
  <button onclick="getHighest()">Show Highest Bid</button>
  <pre id="highest"></pre>
</section>

<script>
  async function getCars() {
    const res = await fetch('/api/cars');
    const data = await res.json();
    document.getElementById('cars').textContent = JSON.stringify(data, null, 2);
  }

  async function registerCar() {
    const carId = document.getElementById('newId').value;

    const model = document.getElementById('newModel').value;
    const owner = document.getElementById('newOwner').value;
    const res = await fetch('/api/register', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ carId, model, owner })
    });
    const data = await res.json();
    document.getElementById('registerResult').textContent = JSON.stringify(data,
null, 2);
  }

  async function placeBid() {
    const carId = document.getElementById('bidCarId').value;
    const bidder = document.getElementById('bidder').value;
    const amount = document.getElementById('bidAmount').value;
    const res = await fetch('/api/bid', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ carId, bidder, amount })
    });
    const data = await res.json();
    document.getElementById('bidResult').textContent = JSON.stringify(data, null,
2);
  }

  async function getHighest() {
    const res = await fetch('/api/highestBid');
    const data = await res.json();
    document.getElementById('highest').textContent = JSON.stringify(data, null, 2);
  }
</script>

```

```
</body>
</html>
```

Ledger.json (empty, so the data will be stored and appear here also after entries)

```
[]
```

Carauction .js

```
import fs from "fs";
const LEDGER_FILE = "./ledger.json";

export class CarAuctionContract {
  constructor() {
    if (!fs.existsSync(LEDGER_FILE)) {
      fs.writeFileSync(LEDGER_FILE, JSON.stringify([]));
    }
  }

  async initLedger() {
    const cars = [
      { carId: "CAR1", model: "Tesla Model 3", owner: "Alice", bid: 0 },
      { carId: "CAR2", model: "BMW X5", owner: "Bob", bid: 0 }
    ];
    fs.writeFileSync(LEDGER_FILE, JSON.stringify(cars, null, 2));
    return "Ledger initialized";
  }

  async queryAllCars() {
    return JSON.parse(fs.readFileSync(LEDGER_FILE));
  }

  async queryCar(carId) {
    const cars = JSON.parse(fs.readFileSync(LEDGER_FILE));
    const car = cars.find(c => c.carId === carId);
    if (!car) throw new Error(`Car ${carId} not found`);
    return car;
  }

  async placeBid(carId, bidder, amount) {
    const cars = JSON.parse(fs.readFileSync(LEDGER_FILE));
    const car = cars.find(c => c.carId === carId);
    if (!car) throw new Error(`Car ${carId} not found`);
    if (parseInt(amount) <= car.bid)
      throw new Error(`Bid must be higher than ${car.bid}`);
    car.bid = parseInt(amount);
    car.owner = bidder;
    fs.writeFileSync(LEDGER_FILE, JSON.stringify(cars, null, 2));
    return car;
  }
}
```

```

async registerCar(carId, model, owner) {
  const cars = JSON.parse(fs.readFileSync(LEDGER_FILE));
  const exists = cars.find(c => c.carId === carId);
  if (exists) throw new Error("Car ID already exists!");
  cars.push({ carId, model, owner, bid: 0 });
  fs.writeFileSync(LEDGER_FILE, JSON.stringify(cars, null, 2));

  return `Car ${carId} registered successfully.`;
}

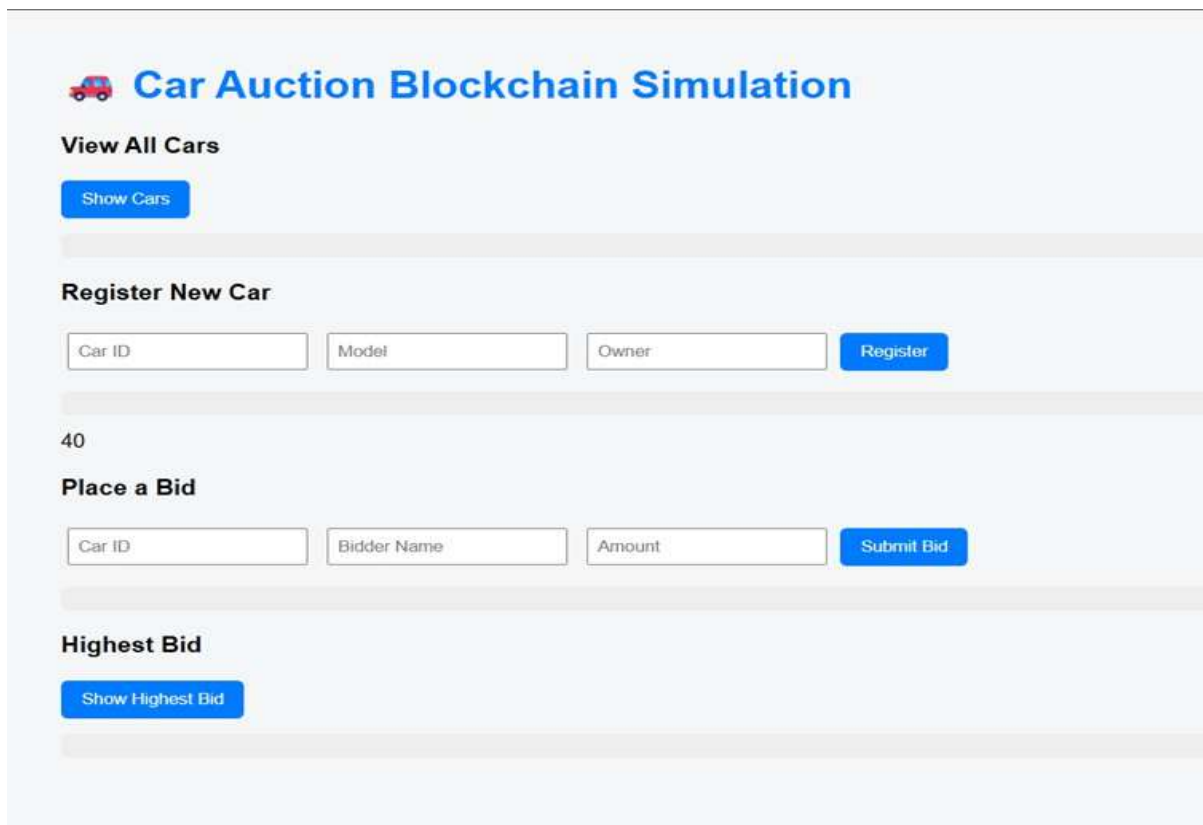
async getHighestBid() {
  const cars = JSON.parse(fs.readFileSync(LEDGER_FILE));
  const topCar = cars.reduce((prev, curr) =>
    prev.bid > curr.bid ? prev : curr
  );
  return topCar;
}

```

Commands to Execute :

- npm install
- node app.js
- Click “Open in New Tab” (top right)
-

Output :



The screenshot displays the 'Car Auction Blockchain Simulation' web application. The interface is light blue with a red car icon. It features several sections: 'View All Cars' with a 'Show Cars' button; 'Register New Car' with input fields for 'Car ID', 'Model', and 'Owner', and a 'Register' button; 'Place a Bid' with input fields for 'Car ID', 'Bidder Name', and 'Amount', and a 'Submit Bid' button; and 'Highest Bid' with a 'Show Highest Bid' button. The number '40' is visible below the 'Register New Car' section.



Car Auction Blockchain Simulation

View All Cars

Show Cars

```
[
  {
    "carId": "CAR1",
    "model": "Tesla Model 3",
    "owner": "Alice",
    "bid": 0
  },
  {
    "carId": "CAR2",
    "model": "BMW X5",
    "owner": "Bob",
    "bid": 0
  },
  {
    "carId": "123",
    "model": "noano",
    "owner": "pp",
    "bid": 260451
  }
]
```

Register New Car

Register

40

Place a Bid

Submit Bid

Highest Bid

Show Highest Bid

```
{
  "carId": "123",
  "model": "noano",
  "owner": "pp",
  "bid": 260451
}
```

This data will also get stored in json format in “**ledger.json**” file

PRACTICAL 7

Aim : Develop an IoT asset tracking app using Block chain. Use an IoT asset tracking device to improve a supply chain by using Block chain, IoT devices, and Node-RED.

Objective :

To develop a simulated IoT Asset Tracking application that integrates IoT devices with a blockchain-like ledger using Node.js and Express on StackBlitz. Students will learn how to track assets, submit sensor data, and maintain an immutable ledger.

It includes :

- **app.js** → Express server + simulated blockchain ledger
- **iotContract.js** → ledger logic
- **iot-ledger.json** → initial empty ledger
- **index.html** → web frontend with form and fetch POST to /track

iotContract.js

```
// iotContract.js
import fs from "fs";
const LEDGER_FILE = "./iot-ledger.json";

export class IoTAssetContract {
  constructor() {
    if (!fs.existsSync(LEDGER_FILE)) {
      fs.writeFileSync(LEDGER_FILE, JSON.stringify([]));
    }
  }

  async addAsset(assetId, location, temperature) {
    const ledger = JSON.parse(fs.readFileSync(LEDGER_FILE));
    const entry = {
      assetId,
      location,
      temperature,
      timestamp: new Date().toISOString()
    };
    ledger.push(entry);
    fs.writeFileSync(LEDGER_FILE, JSON.stringify(ledger, null, 2));
    return entry;
  }

  async getAllAssets() {
    return JSON.parse(fs.readFileSync(LEDGER_FILE));
  }
}
```

iot-ledger.json

```
[]
```

app.js

```
// app.js
import express from "express";
import bodyParser from "body-parser";
import path from "path";
import { fileURLToPath } from "url";
import { IoTAssetContract } from "../iotContract.js";
// Setup __dirname for ES Modules
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const app = express();
app.use(bodyParser.json());
// Serve static files from /public
app.use(express.static(path.join(__dirname, "public")));

const contract = new IoTAssetContract();
// Root route
app.get("/", (req, res) => {
  res.sendFile(path.join(__dirname, "public", "index.html"));
});

// POST /track — add IoT asset
app.post("/track", async (req, res) => {
  try {
    if (!req.body || Object.keys(req.body).length === 0) {
      return res.status(400).json({ error: "Empty or invalid JSON body" });
    }
    const { assetId, location, temperature } = req.body;
    if (!assetId || !location || temperature === undefined) {
      return res.status(400).json({ error: "Missing required fields" });
    }
    const result = await contract.addAsset(assetId, location, temperature);
    res.json(result);
  } catch (err) {
    res.status(500).json({ error: err.toString() });
  }
});

// GET /assets — view all assets
app.get("/assets", async (req, res) => {
  try {
    const result = await contract.getAllAssets();
    res.json(result);
  } catch (err) {
    res.status(500).json({ error: err.toString() });
  }
});

// Start server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`IoT Asset Tracking app running on port ${PORT}`);
});
```

public/index.html

```
<!DOCTYPE html>
<html>
<head>
<title>IoT Asset Tracking</title>
</head>
<body>
<h1>IoT Asset Tracking Simulation</h1>
<form id="trackForm">

<label>Asset ID:</label>
  <input type="text" name="assetId" value="ASSET1" required /><br/><br/>

  <label>Location:</label>
  <input type="text" name="location" value="Warehouse A" required /><br/><br/>

  <label>Temperature:</label>
  <input type="number" name="temperature" step="0.1" value="23.5" required
/><br/><br/>

  <button type="submit">Submit IoT Data</button>
</form>

<h2>Tracked Assets</h2>
<pre id="assetsOutput">Loading...</pre>

<script>
const form = document.getElementById('trackForm');
const output = document.getElementById('assetsOutput');

// Submit IoT data
form.addEventListener('submit', async (e) => {
  e.preventDefault(); // prevent normal form submission

  const data = {
    assetId: form.assetId.value,

    location: form.location.value,
    temperature: parseFloat(form.temperature.value)
  };

  try {
    const response = await fetch('/track', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
    });
    const result = await response.json();
    alert("Data submitted:\n" + JSON.stringify(result, null, 2));
    loadAssets();
  } catch (err) {
```

```

        alert("Error: " + err);
    }
});

// Load all assets
async function loadAssets() {
    try {
        const response = await fetch('/assets');
        const assets = await response.json();
        output.textContent = JSON.stringify(assets, null, 2);
    } catch (err) {

    }
    output.textContent = "Error loading assets: " + err;
}
// Initial load
loadAssets();
</script>
</body>
</html>

```

Output :

IoT Asset Tracking Simulation

Asset ID:

Location:

Temperature:

Tracked Assets

```

[
  {
    "assetId": "ASSET1",
    "location": "Warehouse A",
    "temperature": 23.5,
    "timestamp": "2025-12-06T04:36:07.522Z"
  },
  {
    "assetId": "ASSET2",
    "location": "Warehouse B",
    "temperature": 23.5,
    "timestamp": "2025-12-06T04:36:24.666Z"
  }
]

```

PRACTICAL 8

Aim : Create a global finance block chain application with IBM Block chain Platform Extension for VS Code. Develop a Node.js smart contract and web app for a Global Finance with block chain use case

financeContract.js

```
'use strict';

const { Contract } = require('fabric-contract-api');

class FinanceContract extends Contract {

  async initLedger(ctx) {
    console.log('Initializing Ledger with sample assets...');
    const assets = [
      { id: 'ASSET1', owner: 'BankA', value: 100000, currency: 'USD' },
      { id: 'ASSET2', owner: 'BankB', value: 200000, currency: 'EUR' }
    ];
    for (const asset of assets) {
      await ctx.stub.putState(asset.id, Buffer.from(JSON.stringify(asset)));
    }
  }

  async createAsset(ctx, id, owner, value, currency) {
    const exists = await this.assetExists(ctx, id);
    if (exists) {
      throw new Error(`Asset ${id} already exists`);
    }
    const asset = { id, owner, value: parseFloat(value), currency };
    await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
    return JSON.stringify(asset);
  }

  async readAsset(ctx, id) {
    const assetJSON = await ctx.stub.getState(id);
    if (!assetJSON || assetJSON.length === 0) {
      throw new Error(`Asset ${id} does not exist`);
    }
    return assetJSON.toString();
  }

  async assetExists(ctx, id) {
    const assetJSON = await ctx.stub.getState(id);
    return assetJSON && assetJSON.length > 0;
  }

  async transferAsset(ctx, id, newOwner) {
    const assetJSON = await this.readAsset(ctx, id);
    const asset = JSON.parse(assetJSON);
```

```

    asset.owner = newOwner;

    await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
    return JSON.stringify(asset);
  }

  async getAllAssets(ctx) {
    const allResults = [];
    const iterator = await ctx.stub.getStateByRange("", "");
    let result = await iterator.next();
    while (!result.done) {
      const strValue = Buffer.from(result.value.value.toString()).toString('utf8');
      allResults.push(JSON.parse(strValue));
      result = await iterator.next();
    }
    return JSON.stringify(allResults);
  }
}

module.exports = FinanceContract;

```

app.js

```

const express = require('express');
const bodyParser = require('body-parser');
const app = express();
const port = 3000;
app.use(bodyParser.json());
app.use(express.static('public'));

// Simulated blockchain storage
let blockchain = [];

// API to create asset
app.post('/createAsset', (req, res) => {
  const { id, owner, value, currency } = req.body;
  if (!id || !owner || !value || !currency) return res.status(400).send('Missing fields');

  const exists = blockchain.find(a => a.id === id);
  if (exists) return res.status(400).send('Asset already exists');

  const asset = { id, owner, value, currency, timestamp: new Date().toISOString() };
  blockchain.push(asset);
  res.json(asset);
});

// API to transfer asset
app.post('/transferAsset', (req, res) => {
  const { id, newOwner } = req.body;
  const asset = blockchain.find(a => a.id === id);
  if (!asset) return res.status(404).send('Asset not found');

```

```

asset.owner = newOwner;
res.json(asset);
});
// API to get all assets
app.get('/getAssets', (req, res) => {
res.json(blockchain);
});
app.listen(port, () => console.log(`Server running at http://localhost:\${port}`));

```

index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Global Finance Blockchain</title>
<style>
body { font-family: Arial; margin: 20px; }
input, button { margin: 5px; padding: 5px; }
table { border-collapse: collapse; margin-top: 20px; width: 80%; }
th, td { border: 1px solid #ddd; padding: 8px; text-align: center; }
th { background-color: #f2f2f2; }
</style>

</head>
<body>

<h1>Global Finance Blockchain</h1>

<h3>Create Asset</h3>
<form id="createForm">
  <input type="text" id="assetId" placeholder="Asset ID" required />
  <input type="text" id="owner" placeholder="Owner" required />
  <input type="number" id="value" placeholder="Value" required />
  <input type="text" id="currency" placeholder="Currency" required />
  <button type="submit">Create Asset</button>
</form>

<h3>Transfer Asset</h3>
<form id="transferForm">
  <input type="text" id="transferId" placeholder="Asset ID" required />
  <input type="text" id="newOwner" placeholder="New Owner" required />
  <button type="submit">Transfer</button>
</form>

<h3>All Assets</h3>
<button onclick="loadAssets()">Refresh</button>
<table id="assetTable">
  <thead>
    <tr>

```

```

        <th>Asset ID</th>
        <th>Owner</th>
        <th>Value</th>
        <th>Currency</th>
        <th>Timestamp</th>
    </tr>
</thead>
<tbody></tbody>
</table>

<script>
const createForm = document.getElementById('createForm');
createForm.addEventListener('submit', async e => {
    e.preventDefault();
    const assetId = document.getElementById('assetId').value;
    const owner = document.getElementById('owner').value;
    const value = document.getElementById('value').value;
    const currency = document.getElementById('currency').value;

    const res = await fetch('/createAsset', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ id: assetId, owner, value, currency })
    });
    alert(await res.text());

    loadAssets();
    createForm.reset();
});

const transferForm = document.getElementById('transferForm');
transferForm.addEventListener('submit', async e => {
    e.preventDefault();
    const id = document.getElementById('transferId').value;
    const newOwner = document.getElementById('newOwner').value;

    const res = await fetch('/transferAsset', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ id, newOwner })
    });
    alert(await res.text());
    loadAssets();
    transferForm.reset();
});

async function loadAssets() {
    const res = await fetch('/getAssets');
    const assets = await res.json();
    const tbody = document.querySelector('#assetTable tbody');
    tbody.innerHTML = "";

```

```

assets.forEach(a => {
  tbody.innerHTML += `<tr>
    <td>${a.id}</td>
    <td>${a.owner}</td>
    <td>${a.value}</td>
    <td>${a.currency}</td>
    <td>${a.timestamp}</td>
  </tr>`;
});
}

// Initial load
loadAssets();
</script>

</body>
</html>

```

Output :

Global Finance Blockchain

Create Asset

Transfer Asset

All Assets

Asset ID	Owner	Value	Currency	Timestamp
----------	-------	-------	----------	-----------

Create Asset :

Global Finance Blockchain

Create Asset

Transfer Asset

All Assets

Asset ID	Owner	Value	Currency	Timestamp
123	pp	100	Rs	2026-01-03T14:47:31.057Z

Transfer Asset :

Global Finance Blockchain

Create Asset

Asset ID	Owner	Value
Currency	Create Asset	

Transfer Asset

123	AA	Transfer
----------------------------------	---------------------------------	---

All Assets

Asset ID	Owner	Value	Currency	Timestamp
123	AA	100	Rs	2026-01-03T14:47:31.057Z

PRACTICAL 9

Aim :Develop a voting application using Hyperledger and Ethereum.

Project Structure

hyperledger-voting/

- ├── contracts/
 - └── votingContract.js
- ├── app.js
- ├── package.json
- ├── public/
 - └── index.html

votingContract.js

```
'use strict';
const { Contract } = require('fabric-contract-api');

class VotingContract extends Contract {

  async initLedger(ctx) {
    console.log('Ledger initialized');
    const elections = [];
    await ctx.stub.putState('ELECTIONS',
Buffer.from(JSON.stringify(elections)));
  }

  async createElection(ctx, electionId, description, candidatesJSON) {
    const elections = await this.getElections(ctx);
    const exists = elections.find(e => e.electionId === electionId);
    if (exists) throw new Error(`Election ${electionId} already exists`);

    const candidates = JSON.parse(candidatesJSON).map(c => ({ name: c, votes: 0
}));
    const election = { electionId, description, candidates };
    elections.push(election);
    await ctx.stub.putState('ELECTIONS',
Buffer.from(JSON.stringify(elections)));
    return JSON.stringify(election);
  }

  async vote(ctx, electionId, candidateName) {
    const elections = await this.getElections(ctx);

    const election = elections.find(e => e.electionId === electionId);
    if (!election) throw new Error(`Election ${electionId} not found`);

    const candidate = election.candidates.find(c => c.name === candidateName);
    if (!candidate) throw new Error(`Candidate ${candidateName} not found`);
    candidate.votes += 1;
  }
}
```

```

        await ctx.stub.putState('ELECTIONS',
Buffer.from(JSON.stringify(elections)));
        return JSON.stringify(election);
    }
    async getElections(ctx) {
        const data = await ctx.stub.getState('ELECTIONS');
        if (!data || data.length === 0) return [];
        return JSON.parse(data.toString());
    }
}

module.exports = VotingContract;

```

App.js

```

const express = require('express');
const app = express();
const port = 3000;

// Simulated Hyperledger ledger
let elections = [];

// Middleware
app.use(express.json());
app.use(express.static('public'));

// Create election
app.post('/createElection', (req, res) => {
    const { electionId, description, candidates } = req.body;

    if (!electionId || !description || !candidates) {
        return res.status(400).send('Missing fields');
    }

    const exists = elections.find(e => e.electionId === electionId);
    if (exists) {
        return res.status(400).send('Election already exists');
    }

    const election = {
        electionId,
        description,
        candidates: candidates.map(c => ({
            name: c,
            votes: 0
        })))
    };

    elections.push(election);
    res.json(election);
});

// Vote
app.post('/vote', (req, res) => {
    const { electionId, candidateName } = req.body;

    const election = elections.find(e => e.electionId === electionId);
    if (!election) {

```

```

    return res.status(404).send('Election not found');
  }

  const candidate = election.candidates.find(c => c.name === candidateName);
  if (!candidate) {
    return res.status(404).send('Candidate not found');
  }

  candidate.votes += 1;
  res.json(election);
});

// Get all elections
app.get('/getElections', (req, res) => {
  res.json(elections);
});

// Start server
app.listen(port, () => {
  console.log(`Hyperledger Voting app running at http://localhost:${port}`);
});

```

Index.html

```

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Hyperledger Voting</title>

</head>
<body>
<h1>Voting Application</h1>
<h3>Create Election</h3>
<form id="createForm">
<input type="text" id="electionId" placeholder="Election ID" required />
<input type="text" id="description" placeholder="Description" required />
<input type="text" id="candidates" placeholder="Candidates (comma separated)"
required />
<button type="submit">Create</button>
</form>
<h3>Vote</h3>
<form id="voteForm">
<input type="text" id="voteElectionId" placeholder="Election ID" required />
<input type="text" id="candidateName" placeholder="Candidate Name" required />
<button type="submit">Vote</button>
</form>
<h3>Results</h3>
<button onclick="loadElections()">Refresh</button>
<pre id="results"></pre>
<script>

const createForm = document.getElementById('createForm');
createForm.addEventListener('submit', async e => {
  e.preventDefault();
  const electionId = document.getElementById('electionId').value;
  const description = document.getElementById('description').value;

```

```

const candidates = document.getElementById('candidates').value.split(',');

const res = await fetch('/createElection', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ electionId, description, candidates })
});
alert(await res.text());
loadElections();
});

const voteForm = document.getElementById('voteForm');
voteForm.addEventListener('submit', async e => {
  e.preventDefault();
  const electionId = document.getElementById('voteElectionId').value;
  const candidateName = document.getElementById('candidateName').value;

  const res = await fetch('/vote', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },

body: JSON.stringify({ electionId, candidateName })
  });
  alert(await res.text());
  loadElections();
});
async function loadElections() {
  const res = await fetch('/getElections');
  const elections = await res.json();
  document.getElementById('results').textContent = JSON.stringify(elections, null, 2);
}
</script>
</body>
</html>

```

Output :

Voting Application

Create Election

Vote

Results

Voting Application

Create Election

Election ID

Vote

Results

```
[
  {
    "electionId": "1232",
    "description": "xyz",
    "candidates": [
      {
        "name": "AA",
        "votes": 0
      },
      {
        "name": "PP",
        "votes": 1
      }
    ]
  }
]
```

PART – 2

Voting.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Voting {

    struct Candidate {
        string name;
        uint voteCount;
    }

    mapping(uint => Candidate) public candidates;
    uint public candidatesCount;
    mapping(address => bool) public voters;
    function addCandidate(string memory name) public {
        candidates[candidatesCount] = Candidate(name, 0);
        candidatesCount++;
    }

    function vote(uint candidateId) public {
        require(!voters[msg.sender], "Already voted");
        require(candidateId < candidatesCount, "Invalid candidate");
        voters[msg.sender] = true;
        candidates[candidateId].voteCount++;
    }

    function getCandidate(uint candidateId) public view returns (string memory, uint)
    {
        Candidate memory c = candidates[candidateId];
```

```
return (c.name, c.voteCount);
}
}
```

Index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Ethereum Voting</title>
<script src="https://cdn.jsdelivr.net/npm/web3@1.9.0/dist/web3.min.js"></script>
</head>
<body>
<h1>Ethereum Voting DApp</h1>
<button onclick="addCandidatePrompt()">Add Candidate</button>
<button onclick="votePrompt()">Vote</button>
<button onclick="listCandidates()">List Candidates</button>
<pre id="output"></pre>
<script>
let web3;
let contract;

const contractAddress = 'YOUR_DEPLOYED_CONTRACT_ADDRESS';
const abi = [ /* ABI from Solidity compile */ ];

async function init() {
  if (window.ethereum) {
    web3 = new Web3(window.ethereum);
    await window.ethereum.enable();
    contract = new web3.eth.Contract(abi, contractAddress);
  } else {
    alert("Install MetaMask!");
  }
}

async function addCandidatePrompt() {
  const name = prompt("Candidate name?");
  const accounts = await web3.eth.getAccounts();
  await contract.methods.addCandidate(name).send({ from: accounts[0] });
  alert("Candidate added!");
}

async function votePrompt() {
  const id = parseInt(prompt("Candidate ID?"));
  const accounts = await web3.eth.getAccounts();
  await contract.methods.vote(id).send({ from: accounts[0] });
  alert("Voted!");
}

async function listCandidates() {
  const count = await contract.methods.candidatesCount().call();
  let output = "";
  for (let i = 0; i < count; i++) {
    const c = await contract.methods.getCandidate(i).call();
    output += `ID: ${i}, Name: ${c[0]}, Votes: ${c[1]}\n`;
  }
  document.getElementById('output').textContent = output;
}
```

```
init();  
</script>  
</body>  
</html>
```

Successful connection to Hardhat account :

Connected to Hardhat account: [index.html:352](#)
0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266
>

Output :

Ethereum Voting DApp

Add Candidate Vote List Candidates

Add Candidate :

127.0.0.1:5500 says

Candidate name?

Modi

OK

Cancel

127.0.0.1:5500 says

Candidate added!

OK

Listing Candidate :

Ethereum Voting DApp

Add CandidateVoteList Candidates

ID: 0, Name: ssss, Votes: 1
ID: 1, Name: Modi, Votes: 0

Voting : (Use Candidate ID to vote, e.g 0,1)

127.0.0.1:5500 says

Candidate ID?

OKCancel

127.0.0.1:5500 says

Voted!

OK

Result :

Ethereum Voting DApp

Add CandidateVoteList Candidates

ID: 0, Name: ssss, Votes: 1
ID: 1, Name: Modi, Votes: 1
ID: 2, Name: rahul, Votes: 0

You can see the increment in voting in ID 0 and 1